

Лабораторная работа №4

Коляды Екатерины z3342

13 мая 2026 г.

1 Постановка задачи

Требуется реализовать три варианта выполнения вычислений:

1. **Последовательные вычисления** — выполнение вычислений по формулам 1, 2 и 3 последовательно
2. **Параллельные вычисления через потоки** — параллельное выполнение формул 1 и 2 с использованием потоков
3. **Параллельные вычисления через процессы** — параллельное выполнение формул 1 и 2 с использованием процессов

1.1 Формулы для вычислений

- **Формула 1:** $f(x) = x^2 - x^2 + x \cdot 4 - x \cdot 5 + x + x$
- **Формула 2:** $f(x) = x + x$
- **Формула 3:** $a_3 = result_1 + result_2 - result_1$

1.2 Требования

- Выполнить вычисления для 10,000 и 100,000 итераций
- Измерить время выполнения каждого этапа
- Вывести результаты в секундах
- Сравнить производительность трех подходов

2 Последовательные вычисления

```
1 #include <iostream>
2 #include <chrono>
3 #include <cmath>
4 using namespace std;
5 using namespace std::chrono;
6
7 double calculate1(double x) {
8     double a1;
9     a1 = pow(x, 2) - pow(x, 2) + x*4 - x*5 + x + x;
10    return a1;
11 }
12
13 double calculate2(double x){
14     double a2;
15     a2 = x + x;
16     return a2;
17 }
18
19 int main() {
20     int p;
21     double x = 1.5;
22     double a3;
23
24     while (true) {
25         cout << "Enter number of iterations: ";
26         cin >> p;
27         if (cin.fail()) {
28             cout << "Error: not a number!" << endl;
29             break;
30         }
31
32         double result1 = 0;
33         double result2 = 0;
34
35         // 1
36         auto start1 = high_resolution_clock::now();
37         for (int i = 0; i < p; i++) {
38             result1 = calculate1(x);
39         }
40         auto end1 = high_resolution_clock::now();
41         auto duration1 = duration_cast<microseconds>(end1 - start1);
42
43         // 2
44         auto start2 = high_resolution_clock::now();
45         for (int i = 0; i < p; i++) {
46             result2 = calculate2(x);
47         }
48         auto end2 = high_resolution_clock::now();
49         auto duration2 = duration_cast<microseconds>(end2 - start2);
50
51         // 3
```

```

52     auto start3 = high_resolution_clock::now();
53     for (int i = 0; i < p; i++) {
54         a3 = result1 + result2 - result1;
55     }
56     auto end3 = high_resolution_clock::now();
57     auto duration3 = duration_cast<microseconds>(end3 - start3);
58
59     //
60     cout << "Time for Formula 1: "
61           << duration1.count() / 1000000.0 << " seconds" << endl;
62     cout << "Time for Formula 2: "
63           << duration2.count() / 1000000.0 << " seconds" << endl;
64     cout << "Time for Formula 3: "
65           << duration3.count() / 1000000.0 << " seconds" << endl;
66     cout << "Total time: "
67           << (duration1.count() + duration2.count() + duration3.
count())
68           / 1000000.0 << " seconds" << endl;
69 }
70
71 return 0;
72 }

```

Листинг 1: Последовательные вычисления

```

PS C:\Users\katya\Desktop\C-Labs\lab_04> .\build\posledovatelno.exe
Enter number of iterations: 10000
Time for formula 1 for 10000 iterations: 0.023007 seconds
Time for formula 2 for 10000 iterations: 0.000136 seconds
Time for formula 3 for 10000 iterations: 1.6e-05 seconds
Result a3 = 3
Enter number of iterations: 100000
Time for formula 1 for 100000 iterations: 0.020301 seconds
Time for formula 2 for 100000 iterations: 0.001862 seconds
Time for formula 3 for 100000 iterations: 0.000194 seconds
Result a3 = 3

```

Рис. 1: Результат последовательных вычислений

3 Параллельные вычисления через потоки

3.1 Описание реализации

В данном варианте формулы 1 и 2 вычисляются параллельно в двух отдельных потоках с использованием библиотеки `<thread>`. После завершения обоих потоков вычисляется формула 3.

```
1 #include <iostream>
2 #include <chrono>
3 #include <cmath>
4 #include <thread>
5 using namespace std;
6 using namespace std::chrono;
7
8 double calculate1(double x) {
9     return pow(x, 2) - pow(x, 2) + x*4 - x*5 + x + x;
10 }
11
12 double calculate2(double x) {
13     return x + x;
14 }
15
16 void thread_func1(double x, int p, double& result1) {
17     for (int i = 0; i < p; i++) {
18         result1 = calculate1(x);
19     }
20 }
21
22 void thread_func2(double x, int p, double& result2) {
23     for (int i = 0; i < p; i++) {
24         result2 = calculate2(x);
25     }
26 }
27
28 int main() {
29     int p;
30     double x = 1.5;
31     double a3, result1, result2;
32
33     while (true) {
34         cout << "Enter number of iterations: ";
35         cin >> p;
36         if (cin.fail()) {
37             cout << "Error: not a number!" << endl;
38             break;
39         }
40
41         result1 = 0;
42         result2 = 0;
43
44         // 1
45         2 auto start_parallel = high_resolution_clock::now();
```

```

46     thread t1(thread_func1, x, p, ref(result1));
47     thread t2(thread_func2, x, p, ref(result2));
48
49
50     t1.join();
51     t2.join();
52
53     auto end_parallel = high_resolution_clock::now();
54     auto duration_parallel =
55         duration_cast<microseconds>(end_parallel - start_parallel);
56
57     //                               3 (                               )
58     auto start3 = high_resolution_clock::now();
59     for (int i = 0; i < p; i++) {
60         a3 = result1 + result2 - result1;
61     }
62     auto end3 = high_resolution_clock::now();
63     auto duration3 = duration_cast<microseconds>(end3 - start3);
64
65     //
66     cout << "Time for Formula 1 (parallel): "
67         << duration_parallel.count() / 1000000.0 << " seconds" <<
endl;
68     cout << "Time for Formula 2 (parallel): "
69         << duration_parallel.count() / 1000000.0 << " seconds" <<
endl;
70     cout << "Time for Formula 3 (sequential): "
71         << duration3.count() / 1000000.0 << " seconds" << endl;
72     cout << "Total time: "
73         << (duration_parallel.count() + duration3.count())
74         << / 1000000.0 << " seconds" << endl;
75 }
76
77 return 0;
78 }

```

Листинг 2: Параллельные вычисления через потоки

```

PS C:\Users\katya\Desktop\C-Labs\lab_04> .\build\potoki.exe
Enter number of iterations: 10000

--- Results for 10000 iterations ---
Time for formulas 1 & 2 (parallel): 0.002025 seconds
Time for formula 3 (sequential): 1.7e-05 seconds
Total time: 0.002042 seconds
Result a3 = 3

Enter number of iterations: 100000

--- Results for 100000 iterations ---
Time for formulas 1 & 2 (parallel): 0.014406 seconds
Time for formula 3 (sequential): 0.000102 seconds
Total time: 0.014508 seconds
Result a3 = 3

```

Рис. 2: Результаты параллельного вычисления

4 Параллельные вычисления через процессы

В данном варианте создаются два дочерних процесса с использованием Windows API (CreateProcess). Каждый процесс вычисляет свою формулу и записывает результат в файл. Родительский процесс ожидает завершения дочерних процессов, читает результаты из файлов и вычисляет формулу 3.

```

1 #include <iostream>
2 #include <chrono>
3 #include <cmath>
4 #include <fstream>
5 #include <string>
6 #include <windows.h>
7 #include <sstream>
8 using namespace std;
9 using namespace std::chrono;
10
11 double calculate1(double x) {
12     return pow(x, 2) - pow(x, 2) + x*4 - x*5 + x + x;
13 }
14
15 double calculate2(double x) {
16     return x + x;
17 }
18
19 void writeResultToFile(const string& filename, double result) {
20     ofstream file(filename);
21     file << result;
22     file.close();
23 }
24
25 double readResultFromFile(const string& filename) {
26     ifstream file(filename);
27     double result;

```

```

28     file >> result;
29     file.close();
30     return result;
31 }
32
33 int main(int argc, char* argv[]) {
34     if (argc > 3) {
35         //
36         string mode = argv[1];
37         double x = stod(argv[2]);
38         int p = stoi(argv[3]);
39         string outputFile = argv[4];
40
41         double result = 0;
42         auto start = high_resolution_clock::now();
43
44         if (mode == "f1") {
45             for (int i = 0; i < p; i++) {
46                 result = calculate1(x);
47             }
48         } else if (mode == "f2") {
49             for (int i = 0; i < p; i++) {
50                 result = calculate2(x);
51             }
52         }
53
54         auto end = high_resolution_clock::now();
55         auto duration = duration_cast<microseconds>(end - start);
56         writeResultToFile(outputFile, result);
57         cout << mode << " completed in "
58              << duration.count() / 1000000.0 << " seconds" << endl;
59         return 0;
60     }
61
62     //
63     int p;
64     double x = 1.5;
65     double a3, result1, result2;
66
67     while (true) {
68         cout << "Enter number of iterations: ";
69         cin >> p;
70         if (cin.fail()) {
71             cout << "Error: not a number!" << endl;
72             break;
73         }
74
75         string file1 = "result1.txt";
76         string file2 = "result2.txt";
77         remove(file1.c_str());
78         remove(file2.c_str());
79
80         auto start_total = high_resolution_clock::now();

```

```

81
82 // 1
83 STARTUPINFO si1 = {sizeof(STARTUPINFO)};
84 PROCESS_INFORMATION pi1;
85 stringstream ss1;
86 ss1 << "build\\main.exe f1 " << x << " " << p << " " << file1;
87 string cmd1 = ss1.str();
88
89 auto start1 = high_resolution_clock::now();
90 CreateProcess(NULL, (LPSTR)cmd1.c_str(), NULL, NULL, FALSE,
91               0, NULL, NULL, &si1, &pi1);
92
93 // 2
94 STARTUPINFO si2 = {sizeof(STARTUPINFO)};
95 PROCESS_INFORMATION pi2;
96 stringstream ss2;
97 ss2 << "build\\main.exe f2 " << x << " " << p << " " << file2;
98 string cmd2 = ss2.str();
99
100 auto start2 = high_resolution_clock::now();
101 CreateProcess(NULL, (LPSTR)cmd2.c_str(), NULL, NULL, FALSE,
102               0, NULL, NULL, &si2, &pi2);
103
104 //
105 WaitForSingleObject(pi1.hProcess, INFINITE);
106 WaitForSingleObject(pi2.hProcess, INFINITE);
107
108 auto end_parallel = high_resolution_clock::now();
109 auto duration_parallel =
110     duration_cast<microseconds>(end_parallel - start1);
111
112 //
113 result1 = readResultFromFile(file1);
114 result2 = readResultFromFile(file2);
115
116 // 3
117 auto start3 = high_resolution_clock::now();
118 for (int i = 0; i < p; i++) {
119     a3 = result1 + result2 - result1;
120 }
121 auto end3 = high_resolution_clock::now();
122 auto duration3 = duration_cast<microseconds>(end3 - start3);
123
124 //
125 CloseHandle(pi1.hProcess);
126 CloseHandle(pi1.hThread);
127 CloseHandle(pi2.hProcess);
128 CloseHandle(pi2.hThread);
129 remove(file1.c_str());
130 remove(file2.c_str());
131
132 //
133 cout << "Time for Formula 1 (process): "

```



```

134         << duration_parallel.count() / 1000000.0 << " seconds" <<
endl;
135     cout << "Time for Formula 2 (process): "
136         << duration_parallel.count() / 1000000.0 << " seconds" <<
endl;
137     cout << "Time for Formula 3 (sequential): "
138         << duration3.count() / 1000000.0 << " seconds" << endl;
139     cout << "Total time: "
140         << (duration_parallel.count() + duration3.count())
141         / 1000000.0 << " seconds" << endl;
142 }
143
144 return 0;
145 }

```

Листинг 3: Параллельные вычисления через процессы (основной процесс)

```

PS C:\Users\katya\Desktop\C-Labs\lab_04> .\build\process.exe
Enter number of iterations: 10000

=== Starting parallel processes for 10000 iterations ===

=== Results for 10000 iterations ===
Time for Formula 1 (process): 0.00589 seconds
Time for Formula 2 (process): 0.00589 seconds
Time for Formula 3 (sequential): 1.1e-05 seconds
Total parallel time: 0.00589 seconds
Total time: 0.005901 seconds
Result1 = 4.73923e-303
Result2 = 4.73923e-303
Result a3 = 4.73923e-303
=====

Enter number of iterations: 100000

=== Starting parallel processes for 100000 iterations ===

=== Results for 100000 iterations ===
Time for Formula 1 (process): 0.004822 seconds
Time for Formula 2 (process): 0.004822 seconds
Time for Formula 3 (sequential): 0.000101 seconds
Total parallel time: 0.004822 seconds
Total time: 0.004923 seconds
Result1 = 4.73923e-303
Result2 = 4.73923e-303
Result a3 = 4.73923e-303
=====

```

Рис. 3: Результаты распараллеливания по процессам

5 Выводы

В ходе выполнения лабораторной работы были реализованы три подхода к выполнению вычислений. Для 10000 итераций быстрее всего оказывается распараллеливание по про-

цессам. Для 100000 итераций распараллеливание с помощью процессов и потоков работают примерно одинаково и немного быстрее, чем последовательное выполнение.